SYSTEMS PROGRAMMING | PROJECT STUDY NOTEBOOK

Processes, Pipes, and File Descriptors

A practical Linux systems-programming guide to the Terminator C terminal emulator

WHAT THESE NOTES CONNECT

How a command typed into the X11 GUI becomes a pipeline of child processes; how file descriptors connect each stage; and how EOF, blocking, redirection, and signals emerge from those connections.

Prepared for: revision, implementation walkthroughs, and viva preparation
Project: Terminator modular C terminal emulator
Source baseline: current workspace source inspected for this document
Date: 30 October 2026

This is a teaching document, not a claim that the application is a full POSIX terminal. The project uses an X11 GUI and pipes for command transport; it does not create a pseudo-terminal (PTY).

Contents

1. Learning Objectives . . . 6

2. A Mental Model: Descriptors, Kernel Objects, and Streams . . . 6

2.1 What is a file descriptor? . . . 6

2.2 Standard descriptors . . . 7

2.3 Open file descriptions and fork inheritance . . . 7

3. System Calls: The Small Set That Builds a Shell . . . 7

4. File Descriptors and I/O Redirection . . . 8

4.1 open(): obtain a descriptor for a path . . . 8

4.2 dup2(): replace a standard stream . . . 8

4.3 Input and output redirection . . . 9

4.4 Redirection versus a pipe . . . 9

5. Pipes and Inter-Process Communication . . . 10

5.1 What pipe() creates . . . 10

5.2 Why N commands need N-1 pipes . . . 10

5.3 The all-pipes-upfront strategy used in Terminator . . . 10

5.4 All-upfront versus rolling-pipe construction . . . 11

5.5 Connecting first, middle, and last stages . . . 11

5.6 Reading and writing at the same time . . . 11

5.7 A stage that ignores stdin . . . 12

6. fork(), Scheduling, Blocking, and Backpressure . . . 12

6.1 fork() creates sequentially; pipeline work overlaps . . . 12

6.2 Empty pipe, full pipe, and blocking behavior . . . 12

6.3 Backpressure, not buffer overflow . . . 13

6.4 Process groups and waiting . . . 13

7. Why Closing Every Unused Descriptor Matters . . . 13

7.1 After fork(), both processes have descriptor copies . . . 13

7.2 What goes wrong when a descriptor leaks . . . 14

8. EOF, Ctrl+D, SIGPIPE, and EPIPE . . . 14

8.1 EOF is a condition observed by a reader . . . 14

8.2 EOF macro versus read() returning zero . . . 14

8.3 What Ctrl+D does on a real terminal . . . 14

8.4 SIGPIPE and EPIPE . . . 15

9. Terminator Architecture: GUI, Workers, and Commands . . . 15

9.1 What main() runs . . . 15

9.2 One tab owns one worker and its communication pipes . . . 15

9.3 How shared command history works . . . 16

9.4 The two different kinds of pipe . . . 17

9.5 Why the GUI remains responsive while a worker waits . . . 17

10. Actual Terminator Pipeline Walkthrough . . . 18

10.1 shell_run_line() to execvp() . . . 18

10.2 Pipeline creation and child setup . . . 18

10.3 Parent cleanup and foreground waiting . . . 18

10.4 Foreground and background behavior . . . 18

11. Full Execution Trace: ls | grep ".txt" | sort . . . 19

11.1 Stage graph . . . 19

11.2 Symbolic descriptor map . . . 19

11.3 Timeline . . . 19

12. The Complete Lifecycle as a Diagram . . . 20

12.1 Main thread versus worker read loops . . . 20

13. Other Pipe Case Discussed: multiWatch . . . 20

14. Complete Illustrative C Programs . . . 21

14.1 Parent writes; child reads until EOF . . . 21

14.2 Redirect stdout to a file . . . 23

14.3 A two-stage pipeline . . . 23

14.4 Observe a full-pipe writer block . . . 24

14.5 Observe default SIGPIPE . . . 26

15. Comparison Tables 26

15.1 open() versus pipe() versus dup2() 26

15.2 File redirection versus pipeline communication 27

15.3 Parent process versus child process 27

15.4 Blocking versus nonblocking I/O 27

15.5 EOF versus SIGPIPE 27

15.6 Sequential creation versus concurrent execution 27

15.7 All-pipes-upfront versus rolling pipes 28

16. Common Misconceptions and Corrections 28

17. Quick Revision Notes 28

The six rules worth memorizing 28

18. Viva and Interview Questions 29

Q1. What is a file descriptor? 29

Q2. What does pipe(fds) return? 29

Q3. Why does an N-stage pipeline need N-1 pipes? 29

Q4. Why create pipes before fork()? 29

Q5. What does dup2() do? 29

Q6. What happens if a reader reads an empty pipe? 29

Q7. What happens if a writer writes to a full pipe? 29

Q8. What is the relationship between SIGPIPE and EPIPE? 29

Q9. Why close unused pipe ends? 29

Q10. Are pipeline stages created and executed sequentially? 29

Q11. What decides when a reader stops? 30

Q12. What is the EOF stop condition for a pipe reader? 30

Q13. Does Ctrl+D send EOF to Terminator's child command? 30

Q14. What does the main thread do while a foreground command runs? 30

Q15. Which process receives pipeline stdout? 30

Q16. What status does a pipeline return in this project? 30

Q17. How is multiWatch different from a pipeline? 30

19. Glossary 30

20. Source Map and References 31

20.1 Project source map verified for these notes 31

20.2 Authoritative Linux references 31

1. Learning Objectives

By the end of these notes, you should be able to explain the path from a keypress to a running process, and reason about the descriptors and kernel pipe ends that make a pipeline work.

- Identify a file descriptor as a process-local handle and distinguish it from the kernel object it refers to.
- Explain how open(), pipe(), fork(), dup2(), read(), write(), close(), execvp(), poll(), and waitpid() work together.
- Trace an N-command pipeline and show why it needs N-1 pipes.
- Predict when a read or write blocks, when EOF appears, and when SIGPIPE or EPIPE is produced.
- Explain why all inherited copies of unused pipe ends must be closed.
- Describe which work runs in Terminator's GUI main thread, each tab worker, and each pipeline child.
- Separate actual project behavior from small illustrative programs.

SCOPE

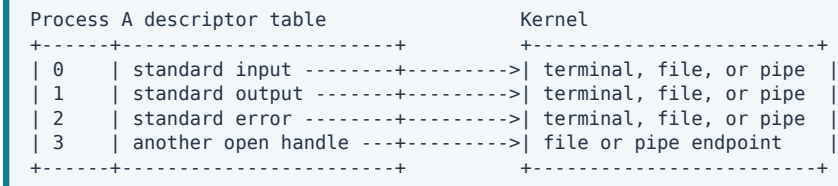
The central example is the project's foreground pipeline. Background jobs, thread/process modes, and multiWatch are included where they clarify descriptor ownership or execution. Future features and unrelated shell topics are outside this notebook.

2. A Mental Model: Descriptors, Kernel Objects, and Streams

2.1 What is a file descriptor?

A file descriptor (FD) is a small non-negative integer used by one process to refer to an open resource. The integer is not the file itself and does not contain the data. The kernel uses the number to find the process's descriptor-table entry, which points to an open kernel object.

MODEL: PROCESS TABLE ENTRY -> KERNEL OPEN OBJECT



The same integer can name different resources in different processes. FD 3 in one process is not automatically FD 3 in another process. After fork(), the child starts with copies of the parent's open descriptor entries, so corresponding entries initially refer to the same underlying open descriptions or pipe endpoints.

DEFINITION

A descriptor is a per-process handle. An open file description is kernel state behind an opened file, including such things as its current offset and file-status flags. Duplicated descriptors and descriptors inherited through fork() can refer to the same open file description.

2.2 Standard descriptors

Descriptor	Name	Usual purpose	Terminator pipeline use
0	stdin	Input stream	Previous pipeline stage, file redirection, or /dev/null for stage 1.
1	stdout	Normal output	Next pipeline pipe, output file, or GUI-output pipe for final stage.
2	stderr	Diagnostics	Project routes pipeline-stage stderr to the tab worker's GUI-output pipe.

The symbolic constants STDIN_FILENO, STDOUT_FILENO, and STDERR_FILENO are normally 0, 1, and 2. Code should use the names rather than scattering the numbers.

2.3 Open file descriptions and fork inheritance

fork() gives the child a separate descriptor table containing copies of the parent's descriptor entries. Those copies refer to the same open file descriptions or pipe endpoint objects. Closing descriptor 4 in the child removes the child's reference; it does not remove descriptor 4 from the parent's table.

DESCRIPTOR TABLES AROUND FORK() (ILLUSTRATIVE FD NUMBERS)

```
Before fork:
Parent FD table                 Kernel object
0 -> terminal input  ----------> open description A
1 -> terminal output ----------> open description B
2 -> terminal errors  ----------> open description C
3 -> pipe read end    ----------> pipe P read endpoint
4 -> pipe write end   ----------> pipe P write endpoint

After fork:
Parent FD table                 Child FD table
0 -> A                           0 -> A
1 -> B                           1 -> B
2 -> C                           2 -> C
3 -> P read ------------------+  3 -> P read
4 -> P write -----------------+  4 -> P write
                              +---- both tables hold references to P
```

The arrow showing a shared reference does not mean that a close() in one process closes the other process's descriptor. The kernel keeps endpoint references until every relevant descriptor copy is closed.

3. System Calls: The Small Set That Builds a Shell

The system calls are the low-level operations. Terminator combines them to connect commands. The return values are part of the design: check a failure before using a descriptor or PID.

Call	Purpose	Success	Failure / note
open(path, flags, mode)	Open or create a filesystem object.	Non-negative FD.	-1; inspect errno. mode is used with O_CREAT and is filtered by umask.
pipe(fds)	Create one anonymous unidirectional byte stream.	0; fds[0] reads and fds[1] writes.	-1; inspect errno.
dup2(old, new)	Make new refer to the same open description as old.	Returns new.	-1; old remains valid. Replacing new is atomic.
fork()	Create a child process.	0 in child; child's PID in parent.	-1 in parent; no child was created.
read(fd, buf, n)	Read up to n bytes.	Positive bytes; 0 means EOF when n > 0.	-1; inspect errno. EINTR may require retry.

Call	Purpose	Success	Failure / note
write(fd, buf, n)	Write up to n bytes.	Number of bytes written; can be partial.	-1; inspect errno. Full blocking pipe waits; no readers may cause SIGPIPE/EPIPE.
close(fd)	Release this process's descriptor reference.	0.	-1; closing a pipe end can change EOF/SIGPIPE behavior.
execvp(file, argv)	Replace current process image with executable.	Does not return on success.	-1 on failure; errno explains why. Searches PATH when file has no slash.
waitpid(pid, &status, opts)	Wait for a child state change.	Returns changed child's PID.	-1 on error; EINTR may require retry.
poll(fds, count, timeout)	Wait until descriptors are ready.	Positive ready count, or 0 on timeout.	-1 on error; may be interrupted.
fcntl(fd, F_SETFL, flags)	Change file status flags, such as O_NONBLOCK.	Command-dependent nonnegative result.	-1 on error.
setpgid(pid, pgid)	Place a process in a process group.	0.	-1 on error.
kill(-pgid, sig)	Send a signal to a process group.	0.	-1 on error; negative pid selects a group.

RETURN-VALUE HABIT

A successful open(), pipe(), or fork() returns a usable resource. A failed call usually returns -1 and sets errno. A successful execvp() is unusual: it never returns because the child is now running another program.

4. File Descriptors and I/O Redirection

4.1 open(): obtain a descriptor for a path

open() asks the kernel to open a path using requested access and creation flags. It returns the lowest currently unused descriptor number in the calling process. For a new output file, O_CREAT requests creation; the mode argument supplies permission bits that are then limited by the process umask.

4.2 dup2(): replace a standard stream

dup2(oldfd, newfd) atomically closes newfd if it is open, then makes newfd refer to the same open file description as oldfd. It does not copy file bytes. It changes where later reads or writes through the selected descriptor go. If oldfd equals newfd, dup2() is a no-op and returns that descriptor.

ILLUSTRATIVE STDOUT REDIRECTION WITH DUP2()

```
int file_fd = open("result.txt",
                   O_WRONLY | O_CREAT | O_TRUNC, 0644);
if (file_fd == -1)
    perror("open");

if (dup2(file_fd, STDOUT_FILENO) == -1)
    perror("dup2");

close(file_fd);       /* fd 1 still refers to result.txt */
printf("saved in the file\n");
```

After dup2(), descriptor 1 is the redirected output. Closing the temporary descriptor returned by open() is important, but it does not close FD 1. They are two descriptor-table entries referring to the same open description.

4.3 Input and output redirection

Shell syntax	What shell setup means	Terminator implementation
command < names.txt	Open file read-only, connect it to FD 0.	redirection_open_input() calls open(path, O_RDONLY), then redirection_apply() dup2s it to stdin.
command > result.txt	Open/create, truncate to length zero, connect to FD 1.	O_WRONLY | O_CREAT | O_TRUNC, then dup2 to stdout.
command >> result.txt	Open/create and append writes at the end.	O_WRONLY | O_CREAT | O_APPEND, then dup2 to stdout.

ACTUAL PROJECT EXCERPT: REDIRECTION_APPLY(), SRC/EXECUTOR.C:32-58

```
if (command->input_path) {
  int fd = redirection_open_input(command->input_path, error_fd);
  if (fd < 0)
    return -1;
  if (dup2(fd, STDIN_FILENO) < 0) {
    int err = errno;
    close(fd);
    errno = err;
    return -1;
  }
  close(fd);
}
if (command->output_path) {
  int fd = redirection_open_output(command->output_path,
                                   command->append, error_fd);
  if (fd < 0)
    return -1;
  if (dup2(fd, STDOUT_FILENO) < 0) {
    int err = errno;
    close(fd);
    errno = err;
    return -1;
  }
  close(fd);
}
```

4.4 Redirection versus a pipe

A file redirection connects a command to a persistent filesystem path. A pipeline connects one process's output stream to the next process's input stream through a temporary kernel buffer. Both use descriptors and dup2(), but the source object is different: a file versus a pipe endpoint.

ORDERING MATTERS

Terminator first connects a pipeline stage's standard input and output to its adjacent pipe ends. Then run_command_in_child() calls redirection_apply(). Therefore an explicit < or > for that command replaces the corresponding pipe connection. Example: in producer | sort < saved.txt, sort reads saved.txt instead of producer's pipe.

Similarly, in producer > saved.txt | consumer, producer's stdout is redirected to the file, so the consumer receives no bytes from that stage and sees EOF once the pipe write ends are closed. The project supports <, >, and >>; this does not imply support for every Bash redirection such as 2>file.

PROJECT DETAIL

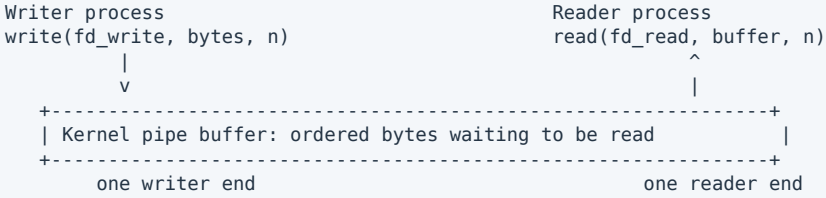
Pipeline setup duplicates output_fd to each child's standard error. The normal stdout of intermediate stages goes to the next stage; stderr goes to the tab's GUI-output pipe. An explicit stdout redirection does not by itself redirect stderr.

5. Pipes and Inter-Process Communication

5.1 What pipe() creates

pipe(fds) creates a unidirectional, anonymous channel held in the kernel. fds[0] is the read end and fds[1] the write end. Bytes written at the write end are queued in a finite kernel buffer and later read from the other end. The channel is a byte stream: it does not preserve read/write message boundaries.

THE PIPE IS A KERNEL QUEUE, NOT A DIRECT MEMORY POINTER



5.2 Why N commands need N-1 pipes

A linear pipeline has one connection between each adjacent pair of commands. N stages create N-1 boundaries, so they need N-1 pipes. The first command has no previous stage, and the last command has no next stage.

THREE STAGES NEED TWO PIPES

```
stage 1: ls       -> [pipe 0] -> grep ".txt" -> [pipe 1] -> sort
                     boundary 1                 boundary 2

N stages:     stage 1 | stage 2 | ... | stage N
Pipe count:       1    +   1    + ... +   1   = N - 1
```

5.3 The all-pipes-upfront strategy used in Terminator

execute_pipeline() allocates an array for count - 1 pairs and creates every internal pipe before it begins the fork loop. Each child can therefore find both the incoming and outgoing pipe in the shared array it inherited. This is easy to trace, at the cost of temporarily holding 2(N-1) descriptors in the worker before the children close unused copies.

ACTUAL PROJECT EXCERPT: CREATE PIPES BEFORE THE FORK LOOP

```c
/* Actual source: src/executor.c:299-311 */
for (size_t i = 0; i + 1 < count; ++i) {
  if (pipe(pipes[i]) < 0) {
    /* report error, close pipes already made, free arrays */
    return 1;
  }
}

/* Fork one child for each command stage. */
for (size_t i = 0; i < count; ++i) {
  pid_t pid = fork();
  /* child connects its standard streams; parent records pid */
}
```

5.4 All-upfront versus rolling-pipe construction

Approach	How it works	Advantages	Care needed
All pipes upfront	Create N-1 pipes, then fork all N stages.	Simple indexed mapping; every stage's descriptors already exist.	More descriptors are open temporarily; all copies must be closed.
Rolling pipe	Keep the previous read end; create the next pipe just before forking the next stage.	Uses fewer pipe descriptors at one time.	More bookkeeping: close the old read end only after the next child inherited it; still launch every stage before waiting.

DEADLOCK TRAP

A rolling design must not wait for the producer to finish before launching the consumer. If the producer fills its pipe first, it blocks waiting for a reader that has not been started. Rolling refers to descriptor reuse, not serial command execution.

5.5 Connecting first, middle, and last stages

For stage i, the incoming pipe is pipes[i - 1] when i is not zero. The outgoing pipe is pipes[i] when i is not the last stage. Terminator uses dup2() to map those ends onto the familiar stdin and stdout descriptors.

Stage	stdin (FD 0)	stdout (FD 1)	stderr (FD 2)
First	Normally /dev/null in this GUI app; replaced by <file if requested.	Write end of pipe 0.	Tab worker output pipe.
Middle i	Read end of pipe i-1.	Write end of pipe i.	Tab worker output pipe.
Last	Read end of pipe N-2.	Tab worker output pipe.	Tab worker output pipe.

ACTUAL PROJECT EXCERPT: CHILD DESCRIPTOR SETUP, SRC/EXECUTOR.C:321-337

```
if (i)
  dup2(pipes[i - 1][0], STDIN_FILENO);
else {
  int nullfd = open("/dev/null", O_RDONLY);
  if (nullfd >= 0) {
    dup2(nullfd, STDIN_FILENO);
    close(nullfd);
  }
}
if (i + 1 < count)
  dup2(pipes[i][1], STDOUT_FILENO);
else
  dup2(output_fd, STDOUT_FILENO);
dup2(output_fd, STDERR_FILENO);

for (size_t j = 0; j + 1 < count; ++j) {
  close(pipes[j][0]);
  close(pipes[j][1]);
}
```

The excerpt is actual project code, with no error checks shown for those particular dup2() calls. The illustrative programs later in this notebook check system-call results more thoroughly.

5.6 Reading and writing at the same time

Yes. One process can be writing while another reads. The kernel buffer decouples their speeds. The CPU scheduler runs runnable processes in turns, on one or more cores. They need not execute at the exact same instant for their I/O to overlap.

5.7 A stage that ignores stdin

A pipe connection does not force a command to read. For pwd | whoami | ls, whoami ignores the bytes from pwd, and ls ignores the bytes from whoami. Their normal stdout still behaves normally: whoami prints the user name into its next pipe, while ls lists the current directory. These short outputs usually fit in the buffer and are simply not consumed.

For a large producer followed by a long-running program that does not read stdin, the pipe can fill and the producer blocks. If that downstream program later exits, all read ends close. A later write by the producer then gets SIGPIPE by default. If the downstream process never exits and never reads, the producer can remain blocked and the foreground pipeline can wait indefinitely.

> **READER STOP CONDITION**
> The reader program decides when it has had enough. It may read until EOF (cat), stop after a requested amount (head -n 1), or stop because of an error or signal. Terminator connects stdin; it does not tell each external program how many bytes to consume.

6. fork(), Scheduling, Blocking, and Backpressure

6.1 fork() creates sequentially; pipeline work overlaps

The worker calls fork() once per stage in a loop. Those system calls happen sequentially in the worker. Crucially, the parent does not wait for stage 1 to finish before forking stage 2. As soon as fork() creates a child, the scheduler may run it. After all children are created, the worker closes its copies of the internal pipe descriptors and waits for the foreground process group.

> **CREATION ORDER IS NOT EXECUTION ORDER**

```
Worker/parent:  fork(ls) ---- fork(grep) ---- fork(sort) ---- close pipes ---- wait
                   |             |              |
                   v             v              v
Children:       ls runs       grep runs       sort runs
                  \___________ pipeline stages overlap ___________/
```

The term concurrent means the stages are in progress during overlapping time. On a multi-core machine they can literally run at the same instant; on one core the scheduler time-slices them. The shell code does not manually schedule byte flow.

6.2 Empty pipe, full pipe, and blocking behavior

Pipe condition	Blocking descriptor behavior	O_NONBLOCK behavior
No bytes available; at least one writer open	read() waits for data or endpoint state to change.	read() returns -1 with EAGAIN/EWOULDBLOCK.
Bytes available	read() returns some available bytes; it may return fewer than requested.	read() returns available bytes without waiting for the requested full count.
Buffer full; at least one reader open	write() waits for room.	write() may return -1 with EAGAIN/EWOULDBLOCK or report a partial write, depending on conditions.
No readers remain	write() generates SIGPIPE; if not terminating by signal, it fails with EPIPE.	Still SIGPIPE/EPIPE semantics; nonblocking does not invent a reader.
No writers remain and buffer drained	read() returns 0, meaning EOF.	read() also returns 0 for EOF.

Terminator's inter-stage pipes are ordinary blocking pipes from pipe(). The GUI's worker-output and status read ends are changed to nonblocking with fcntl(), and the GUI polls them before draining. The two kinds of pipes have different jobs and different I/O settings.

6.3 Backpressure, not buffer overflow

A pipe has finite kernel-managed capacity. When a blocking writer reaches a full buffer, the kernel puts that writer to sleep until a reader frees space. This is backpressure: a slow or absent consumer naturally slows the producer. The kernel does not allow the pipe buffer to overflow into arbitrary memory.

PIPE_BUF IS NOT PIPE CAPACITY

PIPE_BUF is an atomicity limit for writes from multiple writers: writes no larger than PIPE_BUF have specified non-interleaving guarantees. Pipe capacity is the total amount that can be queued before a blocking write must wait. Do not use PIPE_BUF as the pipe's total buffer size.

6.4 Process groups and waiting

Terminator places the stages of one pipeline in a process group. That gives job control a single group identifier to use for Ctrl+C, Ctrl+Z, foreground waiting, and cleanup. waitpid(-pgid, ...) uses a negative pid argument to wait for children in that process group. WUNTRACED also lets the worker detect stopped children.

For a foreground pipeline, wait_for_process_group() keeps reaping group children until there are no more waitable children or a child is stopped. It remembers the last stage's wait status for the shell's pipeline result. This behaves like the common shell convention of reporting the last stage's status; the project does not implement Bash's optional pipefail behavior.

For a background pipeline, execute_pipeline() records the process group as a job and returns to the worker command loop instead of waiting for pipeline completion. The job table stores the group id and last-stage PID for later jobs/fg/bg operations.

FORKING FROM THREAD MODE

In thread mode, execute_pipeline() is called by a tab worker thread in a process that may contain other threads. POSIX/Linux guidance warns that a child of fork() in a multithreaded process should call only async-signal-safe functions before exec(). The current project child path performs additional setup and may run built-ins before execvp(); that is a real portability and robustness consideration for this design, not a pipe-buffer issue.

7. Why Closing Every Unused Descriptor Matters

7.1 After fork(), both processes have descriptor copies

Closing an endpoint in one process does not close another process's copy. The kernel tracks references to the pipe endpoints across every process. EOF and SIGPIPE depend on the total references, not on what one process believes it closed.

CORRECT OWNERSHIP FOR ONE PARENT-TO-CHILD DATA PIPE

```
Immediately after fork():
Parent:  fd 3 = read end       fd 4 = write end
Child:   fd 3 = read end       fd 4 = write end

After assigning jobs:
Parent:  close(fd 3); keep fd 4 only while writing
Child:   keep fd 3 while reading; close(fd 4)

Then parent closes fd 4 after its final write:
Child drains queued bytes, then read(fd 3, ...) returns 0 (EOF).
```

7.2 What goes wrong when a descriptor leaks

Leaked end	How it breaks behavior	Typical symptom
An extra write end remains open	Reader believes a writer might still send bytes, so it does not get EOF after data is drained.	cat or wc waits forever at the end of a pipeline.
An extra read end remains open	Kernel sees a reader even if the intended consumer exited; writer does not get SIGPIPE.	Producer can block on a full pipe with nobody actually consuming.
Any unrelated descriptor remains open	Consumes per-process/system descriptor resources and may keep files/pipes alive across an unexpected fork/exec.	Resource exhaustion, delayed EOF, or confusing child behavior.

A process can also hold multiple duplicates of the same end because of dup2(), dup(), or fork(). EOF appears only after the last write-end reference is closed. SIGPIPE requires that every read-end reference be gone.

THE PROJECT CLEANUP

Each pipeline child dup2s only its intended input/output onto FD 0/1, then closes every original entry in the internal pipes array. The worker parent closes both ends of all internal pipes after forking. This is why a downstream reader can eventually see EOF.

8. EOF, Ctrl+D, SIGPIPE, and EPIPE

8.1 EOF is a condition observed by a reader

EOF means there is no more input available from this stream. It is not a special byte stored at the end of a pipe. For a pipe, read() returns zero after the buffer has been drained and every write-end descriptor has been closed.

EOF ARRIVES AFTER BUFFERED BYTES ARE CONSUMED

```
Pipe state:  [byte 1][byte 2][byte 3]   all write ends are closed
Reader:      read() -> 3 bytes
Reader:      read() -> 0   means EOF
```

8.2 EOF macro versus read() returning zero

Name	Where used	Meaning
EOF macro	C stdio functions such as getchar()/fgetc().	A negative int constant used as a sentinel for end-of-file or stream error; use feof()/ferror() to distinguish.
read() returns 0	POSIX descriptor I/O, when count > 0.	No bytes remain for this read; for a pipe this means its buffer is empty and no write ends remain.
EOF condition	General stream concept.	No more data can be obtained from this stream unless its source changes; it is not itself transmitted as a byte.

8.3 What Ctrl+D does on a real terminal

On a real terminal in canonical mode, Ctrl+D is normally the terminal's VEOF control character. The terminal driver makes pending input available to the waiting program without waiting for a newline. If Ctrl+D is pressed at the start of an empty line, the next terminal read can return zero. It is not a literal byte sent as input, and it does not permanently close the terminal.

TERMINATOR DOES NOT USE A PTY

This application draws an X11 window and sends command lines over its own GUI-to-worker pipe. Ctrl+D is a configured GUI hotkey that closes the active tab when its input line is empty. It does not send the terminal-driver VEOF character to a child. When a tab closes, the GUI closes its input-pipe write end; then worker_run() can observe read(input_fd) == 0.

8.4 SIGPIPE and EPIPE

SIGPIPE is generated when a process writes to a pipe whose read end has no remaining open references. Its default action is to terminate the writing process. If SIGPIPE is ignored or caught, write() instead returns -1 with errno set to EPIPE (after a caught handler returns).

ENDPOINT CLOSURE CHOOSES THE OUTCOME

```
Writer -- write() --> pipe has no readers
                        |
                        +--> default SIGPIPE action: writer terminates
                        |
                        +--> ignored/caught SIGPIPE: write() returns -1, errno=EPIPE

Reader side:
all write ends closed + buffer empty
                        |
                        +--> read() returns 0 (EOF)
```

A full pipe with readers is different: the writer blocks or gets EAGAIN if nonblocking. SIGPIPE is about the absence of readers, not about a full buffer. EOF and SIGPIPE are endpoint-reference consequences viewed from opposite ends.

In Terminator, main() installs handlers for SIGTERM and SIGHUP, but not SIGPIPE. Therefore ordinary pipeline children use the inherited default SIGPIPE disposition. wait_for_process_group() converts a last-stage signal termination into 128 + signal number; on Linux SIGPIPE is signal 13, so a last stage killed by it is reported as 141. If an earlier stage gets SIGPIPE but the final stage exits 0, this project returns the final stage's status.

9. Terminator Architecture: GUI, Workers, and Commands

9.1 What main() runs

main() selects thread or process mode, allocates shared command history, creates the first tab worker, and calls gui_run(). The main thread then stays in the X11 GUI event loop until the window closes. It is not the shell worker's command-reading loop.

ACTUAL HIGH-LEVEL CALL PATH

```
main()
  -> initialize App and SharedHistory
  -> app_add_tab()
  -> gui_run()
       -> poll X11 events and all tab output/status pipes
       -> handle keyboard input and draw the window
  -> app_shutdown()
```

9.2 One tab owns one worker and its communication pipes

Each Tab stores its input/output/status descriptors, worker thread or PID, working directory, input line, history cursor, and scrollback. Clicking a different tab changes App.active_tab. The GUI then sends the next command to that tab's gui_to_worker_fd. Other tab workers remain attached to their own pipes and can keep running while inactive.

ACTUAL STATE LAYOUT FROM INCLUDE/TERMINATOR.H (SHORTENED)

```
typedef struct {
  ExecutionMode mode;
  SharedHistory *history;
  Tab tabs[TERM_MAX_TABS];
  size_t tab_count;
  size_t active_tab;
  int x11_connection_fd;
} App;

typedef struct {
  int gui_to_worker_fd;
  int worker_to_gui_fd;
  int status_from_worker_fd;
  int worker_input_fd;
  int worker_output_fd;
  int worker_status_fd;
  pid_t worker_pid;
  pthread_t worker_thread;
  SharedHistory *history;
  char working_directory[TERM_PATH_BUFFER];
  char input_buffer[TERM_LINE_MAX];
  size_t history_position;
  Scrollback scrollback;
} Tab;
```

App is the application-wide container: it selects the execution mode, points to the shared history, owns the fixed tab array, and remembers which tab is active. Each Tab holds that tab's worker connection, command-editing state, working directory, history cursor, and output scrollback. The active-tab number chooses where future key input goes; it does not move or recreate a worker.

Struct	Beginner mental model	Examples of state
App	The whole terminal window and its tabs.	Mode, tab list, active tab, X11 connection, shared-history pointer.
Tab	One tab plus its worker connection and screen contents.	Pipe descriptors, worker thread/PID, cwd, input buffer, history position, scrollback.
SharedHistory	One shared ring of command strings used by every tab.	Semaphore, entry count, next slot, up to 1024 command lines.

9.3 How shared command history works

History entries are shared by the application, not stored as a separate command list inside each Tab. Each tab does have its own history_position and draft text, so two tabs can browse the same shared list while keeping separate browsing cursors. history_add() saves a non-empty completed command in a bounded ring: when it reaches 1024 entries, new commands replace the oldest ones.

A semaphore protects the ring while a worker writes an entry or a reader copies one. In thread mode, history_init() creates a process-local semaphore and App allocates the history on the heap. In process mode, main() maps the history with MAP_SHARED and initializes a process-shared semaphore, so worker processes see the same entries. The fixed entries array reserves about 8 MiB (1024 x 8192 bytes), in addition to the other application state.

HISTORY BROWSING AND THE HISTORY BUILTIN

```
Completed command -> history_add(shared_history, command)

Up / Down key  -> history_get_recent(shared_history, offset, ...)
                  uses this Tab's history_position

history builtin -> history_snapshot(shared_history, ...)
                   copies a stable oldest-to-newest view under the semaphore
```

WHY MAKE A SNAPSHOT?

The history builtin needs a stable list to print. history_snapshot() briefly locks the shared ring, copies entries into a caller buffer, then unlocks. The caller can print that copy without holding the lock, while another worker safely adds a new command.

Tab pipe	Direction	Owner behavior
Input	GUI -> worker	GUI writes a complete line; worker reads bytes and executes at newline.
Output	Worker/commands -> GUI	Worker and command stdout/stderr write here; GUI polls and appends bytes to that tab's scrollback.
Status	Worker -> GUI	Worker sends status, clear-screen flag, and updated working directory.

THREAD MODE AND PROCESS MODE OWN OPPOSITE PIPE ENDS

```
Thread mode:
GUI thread owns input write, output read, status read
Tab worker thread owns input read, output write, status write
(all live in the same process; descriptor table is process-wide)

Process mode:
GUI parent closes worker ends after fork
Worker child closes GUI ends and unrelated tab ends
GUI parent retains input write, output read, status read
```

9.4 The two different kinds of pipe

Pipe family	Purpose	Reader
Per-tab GUI pipes	Carry submitted commands, display output, and deliver status.	Worker reads input; GUI reads output/status.
Internal pipeline pipes	Connect stdout of one stage to stdin of the next.	Next command stage reads the previous stage's output.
multiWatch capture pipes	Capture each watched command's stdout/stderr separately for polling and forwarding.	The multiWatch supervisor polls and reads each capture pipe.

9.5 Why the GUI remains responsive while a worker waits

gui_run() builds one poll() set containing the X11 connection and each tab's worker-output and status descriptors. poll(..., -1) sleeps until some event is ready. When output arrives, the GUI drains the nonblocking worker-output FD into the corresponding scrollback and redraws. Meanwhile the worker may be blocked in waitpid() for a foreground pipeline.

ACTUAL PROJECT EXCERPTS: INPUT SUBMISSION, WORKER READ, GUI OUTPUT DRAIN

```c
/* gui.c: submit_input() */
send_command_to_worker(tab->gui_to_worker_fd,
                       tab->multiline_buffer, tab->multiline_length);

/* shell.c: worker_run() */
ssize_t bytes_read = read(input_fd, &character, 1);
/* newline completes the command; shell_run_line() executes it */

/* gui.c: gui_run() */
while ((n = read(pf[i].fd, b, sizeof(b))) > 0)
    append_output_text(tab, b, (size_t)n);
```

KEEP THE ROLES SEPARATE

The GUI main thread reads X11 events and worker output. The tab worker reads submitted command bytes. A pipeline stage reads from its internal pipeline stdin. These are separate reads from different file descriptors and often different processes/threads.

10. Actual Terminator Pipeline Walkthrough

10.1 shell_run_line() to execvp()

A complete command line reaches shell_run_line(), which parses it into a Pipeline and expands each stage's arguments. shell_execute() handles certain stateful single-command built-ins in the worker itself. Other commands, and commands inside pipelines, reach execute_pipeline().

A child then runs run_command_in_child(): it changes to the tab's logical working directory, applies file redirections, runs a built-in if recognized, or calls execvp() for an external command. Successful execvp() replaces that child program but preserves the already-configured standard descriptors.

ACTUAL PROJECT EXCERPT: CHILD EXECUTES A COMMAND, SRC/EXECUTOR.C:168-190

```
if (chdir(shell->working_directory) < 0) {
  dprintf(output_fd, "terminator: %s: %s\n",
          shell->working_directory, strerror(errno));
  return 1;
}
if (redirection_apply(command, output_fd) < 0)
  return 1;

int builtin = shell_builtin(shell, command->arguments, STDOUT_FILENO);
if (builtin >= 0)
  return builtin;

if (output_fd > STDERR_FILENO)
  close(output_fd);
execvp(command->arguments[0], command->arguments);
/* If execvp returns, it failed; report errno and return 127. */
```

10.2 Pipeline creation and child setup

execute_pipeline() allocates a PID array and an array of N-1 pipe pairs. It creates all pipes, then forks one child per stage. In each child it sets the process group, copies the shell state needed by the command, maps the correct pipe ends with dup2(), routes stderr to the worker output FD, closes all original internal pipe descriptors, and then invokes run_command_in_child().

10.3 Parent cleanup and foreground waiting

The worker parent records each child PID and joins the stages to a process group. After the fork loop, it closes every internal pipe end. This is not optional: those parent references would otherwise prevent timely EOF or SIGPIPE. For a foreground pipeline, wait_for_process_group() calls waitpid(-pgid, ..., WUNTRACED) until the group's children have completed or one has stopped.

10.4 Foreground and background behavior

Foreground pipeline	Background pipeline
Worker waits for the pipeline process group. GUI main thread still polls and displays output.	Worker records the group as a job and returns to reading the next command. Children continue running.
Last-stage status becomes shell last_status.	Job table later checks state with waitpid(..., WNOHANG | WUNTRACED).
Ctrl+C/Ctrl+Z from GUI targets the active group.	jobs/fg/bg refer to stored process-group ID and job state.

IMPORTANT PARENT DISTINCTION

In thread mode, the command worker is a thread but pipeline stages are child processes created by fork(). In process mode, a tab worker is already a child process, and it creates child processes for its pipeline stages. The X11 GUI remains in the original application process.

11. Full Execution Trace: ls | grep ".txt" | sort

11.1 Stage graph

THREE PROCESSES, TWO INTERNAL PIPES

```
GUI input line
     |
     v
Tab worker parses and expands the line
     |
     +-- fork child 1: ls
     |       stdout -> pipe A write end
     |
     +-- fork child 2: grep ".txt"
     |       stdin  <- pipe A read end
     |       stdout -> pipe B write end
     |
     +-- fork child 3: sort
             stdin  <- pipe B read end
             stdout -> tab worker output pipe -> GUI scrollback
```

11.2 Symbolic descriptor map

For an easy trace, assume the worker-output write descriptor is G and the two internal pipes are A and B. Actual descriptor integers vary by process state; these names stand for returned pipe descriptors, not hard-coded numbers.

Process	FD 0 (stdin)	FD 1 (stdout)	FD 2 (stderr)	Original pipe copies after setup
Worker parent	Worker's inherited input; not pipeline input.	Worker output G.	Worker output or inherited diagnostics.	Closes A.read, A.write, B.read, B.write after all forks.
ls	/dev/null unless redirection.	A.write	G	Closes original A/B descriptors after dup2.
grep	A.read	B.write	G	Closes original A/B descriptors after dup2.
sort	B.read	G	G	Closes original A/B descriptors after dup2.

11.3 Timeline

- 1. Parser makes three SimpleCommand stages and argument expansion runs.
- 2. Worker creates pipe A and pipe B before forking.
- 3. Worker forks ls. The ls child maps FD 1 to A.write and closes original A/B FDs.
- 4. Worker forks grep. The grep child maps FD 0 to A.read and FD 1 to B.write.
- 5. Worker forks sort. The sort child maps FD 0 to B.read and FD 1 to G.
- 6. All stage children are in one process group. They can run before the worker finishes its fork loop.
- 7. Worker closes its own A/B descriptors, so it cannot keep either end alive accidentally.
- 8. ls writes matching candidate names into A. grep reads them, filters lines, and writes matches into B.
- 9. sort reads B and writes sorted text to G. The GUI polls its worker-output read end and appends bytes to that tab's scrollback.
- 10. Worker waits for every child in the group and returns the final stage's status.

IF THE PIPELINE HAS REDIRECTION

For grep ".txt" < list.txt, grep's FD 0 is first mapped to A.read, then redirection_apply() opens list.txt and dup2s it onto FD 0. grep no longer reads A. The previous stage may get SIGPIPE if it writes after the read endpoint closes.

12. The Complete Lifecycle as a Diagram

FROM GUI KEYPRESS TO DISPLAYED RESULT

```
MAIN GUI THREAD                   TAB WORKER                    PIPELINE CHILDREN
---------------                   ----------                    -----------------
X11 KeyPress
  |
edit Tab.input_buffer
  |
press Enter
  |
write GUI->worker pipe ----------> read(input_fd)
                                   newline found
                                   shell_run_line()
                                   parse + expand
                                   execute_pipeline()
                                   pipe() N-1 times
                                   fork stage 1 -----------------> dup2 + execvp
                                   fork stage 2 -----------------> dup2 + execvp
                                   ...                           stages overlap
                                   close parent pipe copies
                                   waitpid(-pgid, ...)
                                   |
poll() returns on output <---------+<----------------------------- stdout/stderr
read worker output pipe
append to tab scrollback
draw window
                                   write WorkerStatus
read status pipe <---------------- status, cwd, clear flag
draw prompt/state
```

The wait at the worker and the GUI poll happen in separate execution contexts. The GUI is not blocked in the worker's waitpid(). This is why output can be displayed while a foreground command is still running.

12.1 Main thread versus worker read loops

Question	Answer for this project
What is running after main() setup?	gui_run() on the main thread, polling the X11 connection and worker output/status pipes.
Who reads the submitted command?	worker_run() in the tab's worker thread or worker child process.
Who reads the internal pipeline?	The next child command in the pipeline, through its stdin descriptor.
Who reads command output for display?	The GUI main thread, from the worker-to-GUI output pipe.
Does switching tabs move a worker?	No. It changes active_tab so future key input goes to a different Tab and its existing input-pipe FD.

13. Other Pipe Case Discussed: multiWatch

multiWatch is not the same as a pipeline. A pipeline streams stage A into stage B. multiWatch launches a list of independent commands, captures each command's output on its own pipe, and forwards ready bytes to the tab output. The supervisor uses poll() rather than busy-waiting, and restarts a finished command after about two seconds.

COMPACT LIST AND INDEPENDENT CAPTURE PIPES

```
multiWatch ["pwd","whoami","ls"]

                +--> command 1 --> capture pipe 1 --+
supervisor ------+--> command 2 --> capture pipe 2 --+--> tab output
                +--> command 3 --> capture pipe 3 --+
```

The project was corrected so a compact bracketed list is split into separate watched command strings. In src/multiwatch.c, add_watch_items() splits on commas outside quote characters. Each watched command has its own output pipe; a poll() loop reads available bytes and the process group lets Ctrl+C stop the watcher.

DO NOT CONFUSE THESE TOPOLOGIES
A three-stage shell pipeline has two pipes arranged in series. Three independent multiWatch commands have three capture pipes arranged in parallel.

14. Complete Illustrative C Programs

The programs in this section are independent learning examples, not copies of Terminator source. Each is compiled by the document-generation check with GCC. They show the kernel interfaces directly and use Linux/POSIX conventions.

14.1 Parent writes; child reads until EOF

The parent closes its unused read end, sends bytes, and closes the final write reference. The child drains queued bytes; its next read returns zero.

PIPE_FORK_EOF.C - CREATE A PIPE, FORK, COMMUNICATE, OBSERVE EOF

```c
#define _POSIX_C_SOURCE 200809L
#include <errno.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/wait.h>
#include <unistd.h>

static int write_all(int fd, const char *data, size_t remaining) {
    while (remaining > 0) {
        ssize_t count = write(fd, data, remaining);
        if (count < 0 && errno == EINTR)
            continue;
        if (count <= 0)
            return -1;
        data += count;
        remaining -= (size_t)count;
    }
    return 0;
}

int main(void) {
    int channel[2];
    if (pipe(channel) == -1) {
        perror("pipe");
        return EXIT_FAILURE;
    }

    pid_t child = fork();
    if (child == -1) {
        perror("fork");
        close(channel[0]);
        close(channel[1]);
        return EXIT_FAILURE;
    }

    if (child == 0) {
        close(channel[1]);
        char buffer[64];

        for (;;) {
            ssize_t count = read(channel[0], buffer, sizeof(buffer));
            if (count == 0)
                break;
            if (count < 0) {
                if (errno == EINTR)
                    continue;
                perror("read");
                _exit(EXIT_FAILURE);
            }
            if (write_all(STDOUT_FILENO, buffer, (size_t)count) == -1) {
                perror("write");
                _exit(EXIT_FAILURE);
            }
        }

        close(channel[0]);
        _exit(EXIT_SUCCESS);
    }

    close(channel[0]);
    const char message[] = "hello through a pipe\n";
    if (write_all(channel[1], message, sizeof(message) - 1) == -1) {
        perror("write");
        close(channel[1]);
        waitpid(child, NULL, 0);
        return EXIT_FAILURE;
    }

    close(channel[1]);  /* The child sees EOF after draining the data. */
    int status;
    if (waitpid(child, &status, 0) == -1) {
        perror("waitpid");
        return EXIT_FAILURE;
    }
    return WIFEXITED(status) ? WEXITSTATUS(status) : EXIT_FAILURE;
}
```

14.2 Redirect stdout to a file

open() obtains a writable descriptor. dup2() makes FD 1 refer to that file. After the original descriptor is closed, printf() still writes through FD 1.

REDIRECT_STDOUT.C - STANDALONE STDOUT REDIRECTION

```c
#define _POSIX_C_SOURCE 200809L
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(void) {
    int file_fd = open("redirected.txt",
                       O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (file_fd == -1) {
        perror("open");
        return EXIT_FAILURE;
    }

    if (dup2(file_fd, STDOUT_FILENO) == -1) {
        perror("dup2");
        close(file_fd);
        return EXIT_FAILURE;
    }
    close(file_fd);  /* fd 1 still refers to the same open file. */

    if (printf("This line is written to redirected.txt\n") < 0) {
        perror("printf");
        return EXIT_FAILURE;
    }
    return EXIT_SUCCESS;
}
```

14.3 A two-stage pipeline

The producer's stdout becomes the pipe write end. The consumer's stdin becomes the read end. The parent closes both ends before waiting, which permits EOF.

TWO_STAGE_PIPELINE.C - PRINTF OUTPUT PIPED INTO WC -L

```c
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <sys/wait.h>
#include <unistd.h>

static void child_error(const char *operation) {
    perror(operation);
    _exit(127);
}

int main(void) {
    int channel[2];
    if (pipe(channel) == -1) {
        perror("pipe");
        return EXIT_FAILURE;
    }

    pid_t producer = fork();
    if (producer == -1) {
        perror("fork producer");
        close(channel[0]);
        close(channel[1]);
        return EXIT_FAILURE;
    }
    if (producer == 0) {
        close(channel[0]);
        if (dup2(channel[1], STDOUT_FILENO) == -1)
            child_error("dup2 producer");
        close(channel[1]);
        execlp("printf", "printf", "alpha\\nbeta\\n", (char *)NULL);
        child_error("exec printf");
    }

    pid_t consumer = fork();
    if (consumer == -1) {
        perror("fork consumer");
        close(channel[0]);
        close(channel[1]);
        waitpid(producer, NULL, 0);
        return EXIT_FAILURE;
    }
    if (consumer == 0) {
        close(channel[1]);
        if (dup2(channel[0], STDIN_FILENO) == -1)
            child_error("dup2 consumer");
        close(channel[0]);
        execlp("wc", "wc", "-l", (char *)NULL);
        child_error("exec wc");
    }

    close(channel[0]);
    close(channel[1]);
    int producer_status;
    int consumer_status;
    if (waitpid(producer, &producer_status, 0) == -1 ||
        waitpid(consumer, &consumer_status, 0) == -1) {
        perror("waitpid");
        return EXIT_FAILURE;
    }
    if (!WIFEXITED(producer_status) || WEXITSTATUS(producer_status) != 0)
        return EXIT_FAILURE;
    return WIFEXITED(consumer_status) ? WEXITSTATUS(consumer_status)
                                      : EXIT_FAILURE;
}
```

14.4 Observe a full-pipe writer block

This Linux-specific example reads the actual capacity with F_GETPIPE_SZ, then asks the writer to send four times that amount while the child deliberately waits before reading. The parent's blocking write cannot finish until the child drains data.

PIPE_BLOCKING.C - BLOCKING BACKPRESSURE DEMONSTRATION

```c
#define _GNU_SOURCE
#include <errno.h>
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/wait.h>
#include <unistd.h>

int main(void) {
    int channel[2];
    if (pipe(channel) == -1) {
        perror("pipe");
        return EXIT_FAILURE;
    }

    int capacity = fcntl(channel[0], F_GETPIPE_SZ);
    if (capacity == -1) {
        perror("F_GETPIPE_SZ");
        close(channel[0]);
        close(channel[1]);
        return EXIT_FAILURE;
    }

    pid_t child = fork();
    if (child == -1) {
        perror("fork");
        close(channel[0]);
        close(channel[1]);
        return EXIT_FAILURE;
    }

    if (child == 0) {
        close(channel[1]);
        sleep(1);  /* Leave the writer alone until the buffer fills. */
        char buffer[4096];
        while (read(channel[0], buffer, sizeof(buffer)) > 0)
            ;
        close(channel[0]);
        _exit(EXIT_SUCCESS);
    }

    close(channel[0]);
    fprintf(stderr, "Pipe capacity: %d bytes; writing four times that amount.\n",
            capacity);
    char block[4096];
    memset(block, 'x', sizeof(block));
    size_t remaining = (size_t)capacity * 4;

    while (remaining > 0) {
        size_t amount = remaining < sizeof(block) ? remaining : sizeof(block);
        ssize_t count = write(channel[1], block, amount);
        if (count < 0 && errno == EINTR)
            continue;
        if (count <= 0) {
            perror("write");
            close(channel[1]);
            waitpid(child, NULL, 0);
            return EXIT_FAILURE;
        }
        remaining -= (size_t)count;
    }

    close(channel[1]);
    fprintf(stderr, "Writer completed after the reader began draining.\n");
    int status;
    if (waitpid(child, &status, 0) == -1) {
        perror("waitpid");
        return EXIT_FAILURE;
    }
    return WIFEXITED(status) ? WEXITSTATUS(status) : EXIT_FAILURE;
}
```

14.5 Observe default SIGPIPE

The child closes its only read-end descriptor and writes. The default SIGPIPE action terminates that child; the parent detects the signal using waitpid().

SIGPIPE_DEMO.C - WRITER WITH NO READERS

```c
#define _POSIX_C_SOURCE 200809L
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/wait.h>
#include <unistd.h>

int main(void) {
    int channel[2];
    if (pipe(channel) == -1) {
        perror("pipe");
        return EXIT_FAILURE;
    }

    pid_t child = fork();
    if (child == -1) {
        perror("fork");
        close(channel[0]);
        close(channel[1]);
        return EXIT_FAILURE;
    }

    if (child == 0) {
        close(channel[0]);
        write(channel[1], "x", 1);  /* No process keeps a read end open. */
        _exit(0);                   /* Normally unreachable: SIGPIPE ends us. */
    }

    close(channel[0]);
    close(channel[1]);
    int status;
    if (waitpid(child, &status, 0) == -1) {
        perror("waitpid");
        return EXIT_FAILURE;
    }
    if (WIFSIGNALED(status) && WTERMSIG(status) == SIGPIPE) {
        puts("Child terminated by SIGPIPE.");
        return EXIT_SUCCESS;
    }
    puts("Child did not receive the default SIGPIPE action.");
    return EXIT_FAILURE;
}
```

15. Comparison Tables

15.1 open() versus pipe() versus dup2()

Operation	Creates / changes	Data source or destination	Typical shell use
open()	A descriptor for a path.	Regular file or other named filesystem object.	Open file for <, >, >>.
pipe()	Two descriptors and a kernel pipe channel.	Anonymous read/write endpoints.	Connect adjacent process stages or GUI/worker roles.
dup2()	One descriptor-table mapping.	Does not move bytes; reroutes future I/O.	Install pipe/file on stdin or stdout.

15.2 File redirection versus pipeline communication

Property	File redirection	Pipeline
Object	Filesystem path / open file description.	Kernel pipe buffer and two endpoints.
Lifetime	File data may persist after the command.	Temporary channel exists while references remain open.
Connection	open() then dup2() to FD 0 or FD 1.	pipe(), fork(), dup2() adjacent ends.
Flow	< reads a file; > truncates/writes; >> appends.	Writer streams bytes to a reader.
End condition	File offset reaches end for reads.	All writers close and buffer drains -> EOF.

15.3 Parent process versus child process

Parent / worker	Pipeline child
Creates pipe arrays and calls fork() for each stage.	Inherits descriptors at fork and receives its own descriptor table.
Closes internal pipe ends after launching all stages.	Maps its incoming/outgoing endpoint to FD 0/1 with dup2().
Sets/records process group and PIDs.	Runs built-in in child or replaces itself through execvp().
Foreground worker waits using waitpid(); background worker records a job.	Runs command concurrently; may block reading/writing its pipe.

15.4 Blocking versus nonblocking I/O

Mode	When no data / no space	How the project uses it
Blocking	read on empty open pipe or write on full pipe sleeps until progress or endpoint change.	Inter-stage pipeline pipes from pipe().
Nonblocking	Would-block operation returns EAGAIN/EWOULDBLOCK instead of sleeping.	GUI sets worker output/status read FDs O_NONBLOCK and drains them after poll().
poll + nonblocking	poll waits for readiness; nonblocking read drains available bytes without risking a long GUI stall.	X11 loop handles both window and worker events.

15.5 EOF versus SIGPIPE

Question	EOF	SIGPIPE
Observed by	Reader.	Writer.
Condition	No write-end references remain and queued data is drained.	No read-end references remain when a write is attempted.
Typical outcome	read() returns 0.	Signal generated; default action terminates writer.
Error alternative	Not a read error for a nonzero requested count.	If ignored/caught, write() returns -1 with EPIPE.

15.6 Sequential creation versus concurrent execution

What happens	Order / concurrency
The parent calls fork() in a for-loop.	Creation calls happen sequentially.
A child becomes runnable after fork().	The scheduler may run it immediately, even while the parent prepares the next stage.
The parent doesn't wait between stage forks.	Stage execution overlaps.
All foreground stages eventually finish.	Worker reaps them with waitpid() after the fork loop.

15.7 All-pipes-upfront versus rolling pipes

Topic	All upfront	Rolling
Pipe descriptors	All N-1 pairs exist before first fork.	Only previous read end and next pair need be held.
Mapping	Child indexes into pipes[i-1] and pipes[i].	Parent passes current endpoints as loop state.
Complexity	Easy to inspect; more temporary FDs.	Fewer FDs; more close/ownership bookkeeping.
Concurrency rule	Fork all stages, then wait.	Also fork all stages, then wait; never wait after each producer.

16. Common Misconceptions and Corrections

Misconception	Correction
A pipe is an unlimited buffer.	It is finite. A blocking writer sleeps when full; this is backpressure.
PIPE_BUF is the buffer capacity.	PIPE_BUF concerns atomicity between writers; total pipe capacity is a different kernel property.
One process closing a descriptor closes every copy.	close() affects only that process's descriptor entry; other forked/duplicated entries remain.
An empty pipe always means EOF.	If any writer is still open, a blocking reader waits for more data. EOF requires no writers and an empty buffer.
The producer always waits for the consumer to finish.	The producer and consumer overlap; reads/writes block only when the buffer state requires it.
A command connected to a pipe must read from it.	No. A program may ignore stdin or exit early; the producer can fill/block or later get SIGPIPE.
The GUI main thread reads commands from the worker.	The worker reads command bytes; the GUI reads X11 events and worker output/status.
Ctrl+D in Terminator is the same as terminal EOF.	This project maps Ctrl+D to closing the active empty tab. There is no PTY line discipline.
waitpid() carries pipeline output.	waitpid() reports child state. Output travels through file descriptors and pipes.
forking stages one-by-one means they execute one-by-one.	Only creation is sequential; children can run concurrently because the parent does not wait between forks.
stderr goes to the next pipeline stage.	In this project child stderr is routed to the tab output pipe, while stdout follows pipeline connections.
A pipe preserves one write as one read.	A regular pipe is a byte stream. Readers may receive partial or combined writes.

17. Quick Revision Notes

> **THIRTY-SECOND EXPLANATION**
> A file descriptor is a per-process integer handle. pipe() gives a read end and a write end. fork() copies descriptors into a child. dup2() puts the desired pipe/file on stdin or stdout. Close unused copies so EOF and SIGPIPE can occur. The kernel buffers a finite number of bytes: reads block on empty pipes with live writers, writes block on full pipes with readers. Terminator's worker launches all pipeline children before waiting; the GUI main thread independently polls X11 and worker output.

The six rules worth memorizing

- N pipeline stages need N-1 inter-stage pipes.
- pipefd[0] reads; pipefd[1] writes.

- Create each pipe before the child that must inherit it.
- dup2(source, STDIN_FILENO/STDOUT_FILENO) sets the command's stream.
- Close every pipe descriptor copy that a process does not need.
- read() returns 0 for EOF only after buffered data drains and no writers remain.
- Full blocking pipe -> writer waits. No readers -> SIGPIPE/EPIPE.
- Fork calls are sequential; pipeline work is concurrent because there is no wait between stage forks.
- The GUI main thread reads output; the tab worker reads the submitted command.
- The project uses X11 and pipes rather than a PTY; its Ctrl+D hotkey closes an empty tab.

18. Viva and Interview Questions

Q1. What is a file descriptor?

Answer: It is a small integer in one process's descriptor table. The kernel uses it to find an open file description or another I/O endpoint, such as a pipe.

Q2. What does pipe(fds) return?

Answer: It returns 0 on success and stores two descriptors: fds[0] is the read end and fds[1] is the write end. On failure it returns -1 and sets errno.

Q3. Why does an N-stage pipeline need N-1 pipes?

Answer: Each pipe connects one adjacent pair. A linear chain of N stages has N-1 boundaries.

Q4. Why create pipes before fork()?

Answer: A child inherits the descriptors open at fork time. Creating the pipe first gives both the parent and child access to the endpoints they must set up.

Q5. What does dup2() do?

Answer: It atomically replaces the descriptor numbered newfd with a copy of oldfd's reference. It routes future I/O through newfd; it does not copy data.

Q6. What happens if a reader reads an empty pipe?

Answer: With blocking I/O and at least one writer open, read() sleeps until data or endpoint closure. If no writers remain and the buffer is empty, it returns 0.

Q7. What happens if a writer writes to a full pipe?

Answer: With blocking I/O and a reader still open, write() sleeps until space is available. With O_NONBLOCK, the call may return EAGAIN instead.

Q8. What is the relationship between SIGPIPE and EPIPE?

Answer: A write with no readers generates SIGPIPE. Its default action terminates the writer. If the signal is ignored or caught, write() fails with errno EPIPE.

Q9. Why close unused pipe ends?

Answer: EOF is visible only when all write-end copies close. SIGPIPE is possible only when all read-end copies close. A stray descriptor can keep the other process waiting or prevent the expected signal.

Q10. Are pipeline stages created and executed sequentially?

Answer: The parent calls fork() sequentially, but it does not wait after each fork. Each child can run while the parent starts later stages, so the pipeline stages overlap.

Q11. What decides when a reader stops?

Answer: The reader program. It can stop on EOF, after enough data, or because of an error or signal.

Q12. What is the EOF stop condition for a pipe reader?

Answer: Its read() returns 0 after the pipe has no queued bytes and all write-end descriptors are closed.

Q13. Does Ctrl+D send EOF to Terminator's child command?

Answer: No. Terminator has a custom X11 key binding and no PTY. Ctrl+D closes the active tab when its input line is empty. Closing the tab closes the GUI to worker pipe; the worker can then read 0.

Q14. What does the main thread do while a foreground command runs?

Answer: It stays in gui_run(), polling X11 and all worker output/status pipes. The tab worker waits for pipeline children; the GUI remains available to display output and handle input.

Q15. Which process receives pipeline stdout?

Answer: Each non-final stage's stdout is the next internal pipe's write end. The final stage's stdout is the tab worker's output pipe, which the GUI reads.

Q16. What status does a pipeline return in this project?

Answer: The worker waits for the process group and stores the last stage's exit/signal status. A failure in an earlier stage does not automatically override a successful last stage; pipefail is not implemented.

Q17. How is multiWatch different from a pipeline?

Answer: A pipeline is a serial data chain. multiWatch starts independent commands concurrently, gives each a capture pipe, polls output, and restarts commands after they finish.

19. Glossary

Term	Meaning
Blocking I/O	A read/write call that sleeps until it can make progress or endpoint state changes.
Backpressure	A slow consumer causing a producer to slow or block because a finite buffer is full.
Descriptor table	Per-process mapping from small FD numbers to kernel references.
EOF	End-of-stream condition; for a drained pipe with no writers, read() returns 0.
EPIPE	write() error indicating there are no readers, visible when SIGPIPE does not terminate the caller.
File descriptor	Per-process integer handle used by read(), write(), dup2(), close(), and related calls.
Open file description	Kernel object with state for one open instance, such as a file offset and status flags.
PIPE_BUF	Pipe write atomicity threshold; not the total pipe capacity.
Process group	Related processes grouped for job-control signals and wait operations.
SIGPIPE	Signal generated when a process writes to a pipe with no open readers.
VEOF	Terminal control character, usually Ctrl+D, interpreted by a real terminal line discipline.
Worker	Per-tab shell loop that reads GUI command bytes, invokes shell_run_line(), and sends output/status back.

20. Source Map and References

20.1 Project source map verified for these notes

Project file	Relevant responsibility	Key lines
src/main.c	Initialize App/history/tabs; select mode; call GUI loop; shutdown.	15-64
src/tab.c	Create per-tab pipes; launch worker thread or process; close descriptors.	40-163
src/shell.c	worker_run() reads command input, executes complete lines, writes status.	204-261
src/history.c	Shared ring buffer; semaphore locking; recent-entry lookup and snapshots.	6-72
src/gui.c	Submit command; key handling; poll X11 and worker pipes; display output.	257-314, 315-421, 423-596
src/executor.c	open/dup2 redirection; child launch; pipeline pipes; process groups; waitpid.	17-58, 168-191, 238-395, 485-555
src/multiwatch.c	Split command list; one capture pipe per command; poll, forward, restart.	26-233
include/terminator.h	App and Tab state, pipe descriptors, worker APIs.	29-92
README.md	No-PTY behavior and user-facing controls.	47-75

20.2 Authoritative Linux references

The behavioral descriptions of Linux system calls, pipes, signals, and terminal line discipline were checked against the Linux man-pages project. The project-specific descriptions above were checked against the source map in the previous table.

1. pipe(7) - Pipes and FIFOs
2. pipe(2) - Create pipe
3. fork(2) - Create child process
4. dup(2) - Duplicate a file descriptor
5. open(2) - Open and possibly create a file
6. read(2) - Read from a file descriptor
7. write(2) - Write to a file descriptor
8. wait(2)/waitpid(2) - Wait for child state
9. exec(3) - Execute a program
10. termios(3) - Terminal I/O settings and VEOF
11. signal(7) - Signal overview and default actions

FINAL MODEL

The kernel owns the pipe buffer and endpoint reference counts. The project owns the descriptor wiring, process launch order, close discipline, process groups, waiting, and GUI event loop. Understanding both layers explains the observed behavior.